

大语言模型技术方法与应用

课程总报告

从 0 到 1 训练一个属于自己的小型语言模型：Tokenizer → Pretrain → SFT → DPO 全链路实践

姓名	廖佩瑶	学号	2022K8009915031
课程	大语言模型技术方法与应用	日期	2026 年 5 月
框架	MiniMind (类 LLaMA Decoder-only)	平台	SCNet 云平台 · NVIDIA RTX 4090
作业一仓库	https://github.com/qing121201/Tokenizer_Project		
作业二/三仓库	https://github.com/qing121201/MiniMind_Pretrain		

一、引言：从 0 到 1 的炼丹之旅

本学期围绕“如何从零训练一个可用的语言模型”这一主题，依次完成了三次实践作业：

Tokenizer 训练 → **预训练 (Pretrain)** → **监督微调与偏好对齐 (SFT + DPO)**。三次作业之间并非孤立的练习，而是构成了一条完整的“**全链路炼丹流水线**”——作业一为模型提供“语言的最小语义单元”，作业二让模型学会“说话”，作业三让模型学会“听话并说人话”。本报告将三次作业成果串联整合，回顾整个生命周期，并分享我在过程中遇到的最大工程阻碍及解决方案。

全链路总览：

- Tokenizer 阶段**：使用 BPE 算法在 127 万条文本上训练得到 6400 词表的分词器；
- Pretrain 阶段**：基于该 Tokenizer，从零训练 25.83M 参数的 Decoder-only 模型，Loss 从 7.64 降至 2.20；
- SFT 阶段**：使用 7.5 万条对话数据进行指令微调，模型摆脱“无脑续写”，具备指令跟随能力；

④ **DPO 阶段**：基于人类偏好对 (chosen/rejected) 进一步对齐，reward margin 从 0.0049 增长至 0.0627，回答更结构化、定位更清晰。

二、第一站：Tokenizer 训练（作业一）

2.1 任务定位

Tokenizer 是大模型与文本世界之间的“翻译官”。本阶段目标是从零构建一个可同时处理中英文的 BPE 分词器，为后续预训练提供 token 化基础。

2.2 关键设置

参数	取值	说明
算法	BPE (Byte Pair Encoding)	字节对编码
词表大小	6400	小词表，匹配小模型规模
预分词器	ByteLevel	字节级、不存在 OOV
归一化器	NFKC	Unicode 标准化
特殊 Token	<pad>, <unk>, <s>, </s>	id = 0/1/2/3
训练数据	pretrain_t2t_mini.jsonl	1,270,238 行清洗后语料

2.3 实验结果

测试文本类型	Token 数	字符数	压缩率
英文	17	44	0.3864 (最优)
中文	21	41	0.5122
中英混合	21	44	0.4773

关键观察：由于训练数据以中文为主，“人工智能”“计算机科学”等高频中文词组被合并为单 token；而英文 token 多保留 BPE 子词形式（如 'quick' → 'Ġqu' + 'ick'）。ByteLevel BPE 完美解决了 OOV 问题，但 6400 的小词表导致中文压缩率偏低——这一限制将在后续 Pretrain 阶段直接影响模型对长文本的有效利用。

三、第二站：Pretrain 预训练（作业二）

3.1 模型架构

参数	取值	说明
架构	Decoder-only Transformer	类 LLaMA: RMSNorm + RoPE + SwiGLU
hidden_size	512	隐藏维度
num_hidden_layers	8	Transformer 层数
num_attention_heads	8	注意力头数
max_position_embeddings	512	上下文长度
总参数量	25.83 M	Small 规模

3.2 训练配置

超参	取值
Batch Size	32
Learning Rate	5e-4 (cosine 调度)
Epochs	1
训练总步数	39,695 步
优化器	AdamW
损失函数	Cross-Entropy (标准 LM Loss)

3.3 训练曲线分析

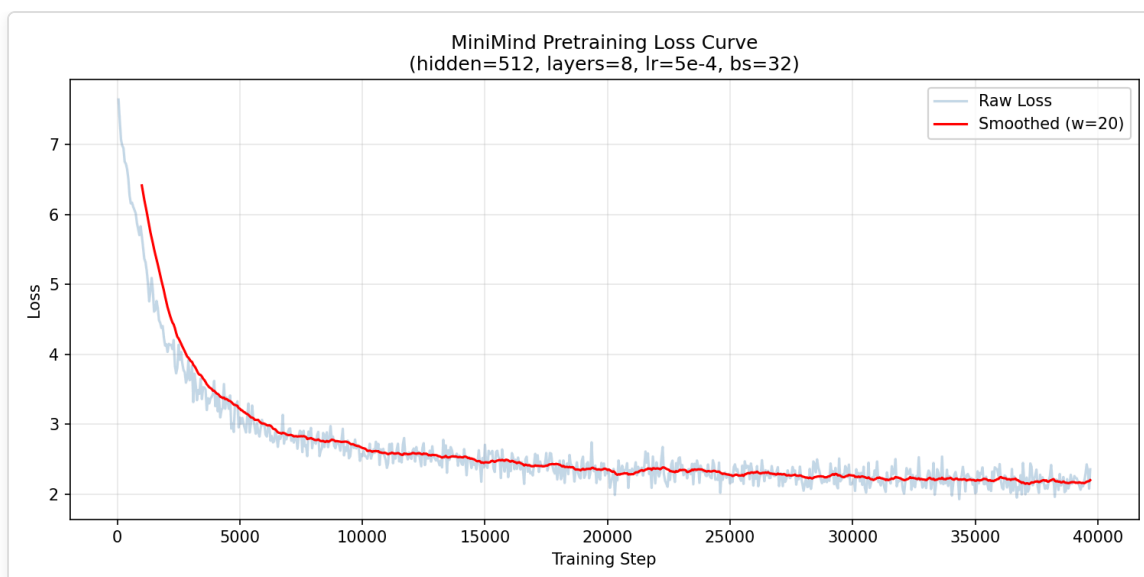


图 1: Pretrain Loss 曲线 (hidden=512, layers=8, lr=5e-4, bs=32)

三阶段收敛性总结:

1. **急速下降期 (0~2000 步)** : Loss 从 7.6 降至 3.5, 模型学习 token 频率分布与基本语言统计规律;
2. **稳步下降期 (2000~15000 步)** : Loss 从 3.5 降至 2.5, 开始学习更复杂的句法与语义关系;
3. **平稳收敛期 (15000~39695 步)** : Loss 在 2.2~2.4 间小幅波动, 已达 1 epoch 的收敛态。

3.4 续写能力评估

输入: 北京是中国的首都,

输出: 北京是中国的首都, 而张家界的领袖, 则是中国的皇家界领袖, 如汉朝、宋、元等..... (出现明显复读循环)

预训练后的模型已学会基本词组搭配与中英语序, 能够自然续写, 但典型表现是: "**会说话, 但不一定说对, 且容易复读**". 这正是后续 SFT 与 DPO 要解决的核心问题。

3.5 Scaling Law 雏形思考

本实验使用 25.83M 参数的小模型在 127 万条数据上训练 1 epoch, 最终 Loss 收敛于约 2.2。根据 Scaling Law (Kaplan et al., 2020; Chinchilla, 2022) 的经验规律, 模型 Loss 与参数量 N 、数据量 D 满足近似幂律关系:

$$L(N, D) \approx A \cdot N^{-\alpha} + B \cdot D^{-\beta} + L_{\infty}$$

预期若模型层数从 8 加倍至 16（参数量约 50M），最终 Loss 会再下降 0.1~0.3，但训练时间和显存开销也近似翻倍。在受限算力下，**提升数据质量与多样性**往往比单纯加深网络更具收益。

四、第三站：SFT 监督微调 + DPO 偏好对齐（作业三）

4.1 SFT 阶段

SFT 使用 `sft_mini_512.jsonl`（约 75,921 条对话），采用 MiniMind 标准 Chat Template：

```
<s>[INST] 用户输入 [/INST] 助手回答 </s>
```

训练时**仅对 assistant 回答部分计算 loss**，用户输入部分被 mask 掉。SFT 损失函数为标准的 Next Token Prediction：

$$\mathcal{L}_{\text{SFT}} = - \sum_{t \in \text{assistant}} \log P_{\theta}(y_t | y_{<t}, x)$$

参数	SFT	DPO
学习率	1e-5	1e-6
批大小	16	4
训练轮数	1	1
最大序列长度	512	1024
KL 系数 β	—	0.1 / 0.5 / 0.9（消融）

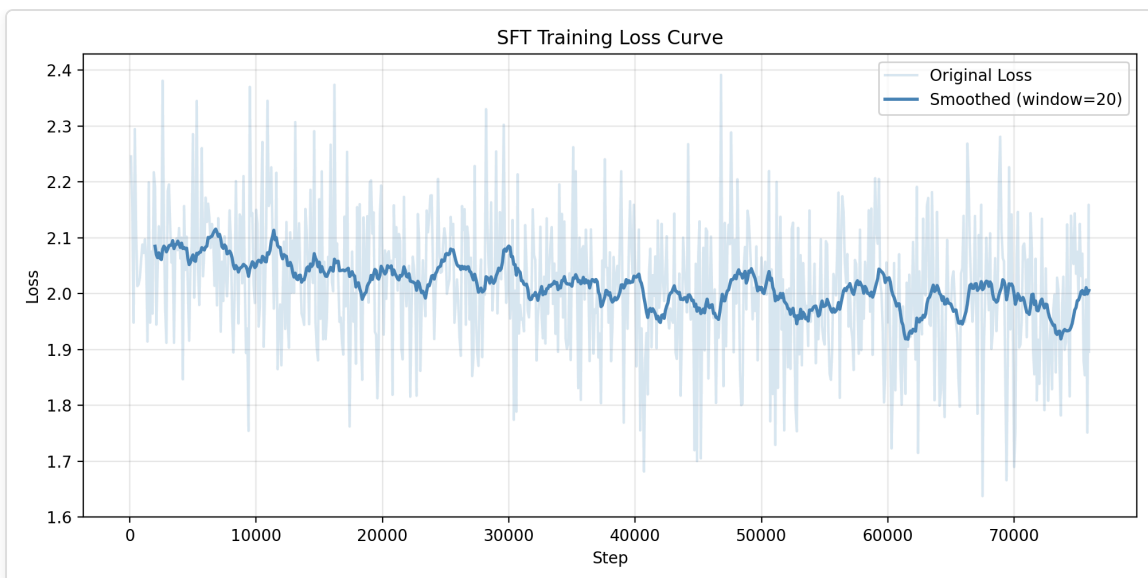


图 2: SFT 训练 Loss 曲线 (浅色为原始 loss, 深色为滑动平均 window=20)

4.2 DPO 阶段

DPO (Direct Preference Optimization) 不需要显式奖励模型, 直接利用 chosen/rejected 偏好对优化策略模型, 同时维护冻结的参考模型 π_{ref} 以 KL 散度约束防止偏离 SFT 太远。损失函数为:

$$\mathcal{L}_{\text{DPO}} = -\mathbf{E}_{(x, y_w, y_l) \sim D} \left[\log \sigma \left(\beta \log \frac{\pi_{\theta}(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \beta \log \frac{\pi_{\theta}(y_l|x)}{\pi_{\text{ref}}(y_l|x)} \right) \right]$$

其中 y_w 表示 chosen 回答, y_l 表示 rejected 回答, β 为 KL 惩罚强度系数, σ 为 sigmoid 函数。

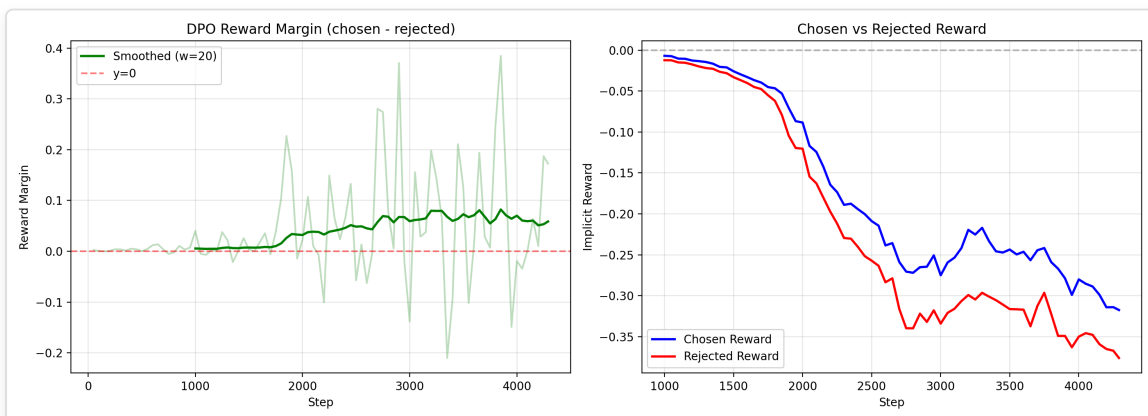


图 3: DPO Reward Margin 变化曲线 (左: margin; 右: chosen vs rejected reward)

指标	数值
前 1/4 步 reward margin 平均	0.0049
后 1/4 步 reward margin 平均	0.0627
整体最大值 / 最小值	0.3848 / -0.2102
margin > 0 的比例	75.6%

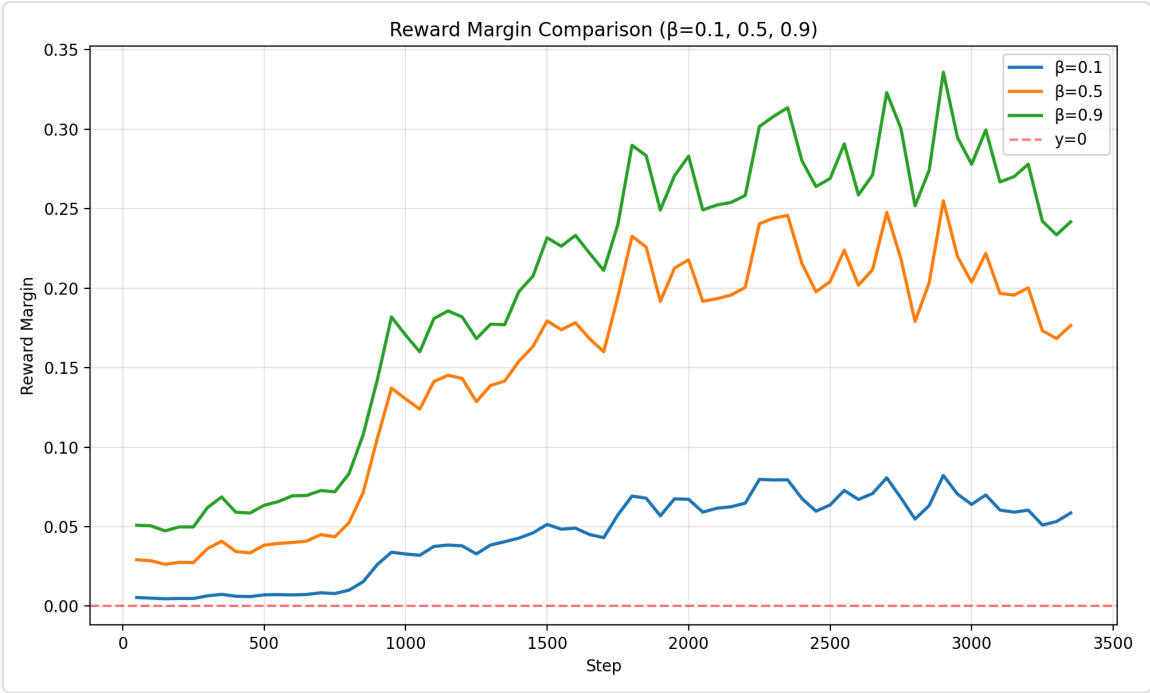


图 4：不同 β 参数下的 Reward Margin 对比曲线 ($lr=1e-6, 1 \text{ epoch}$)

β 值	KL 惩罚	偏好对齐能力	遗忘风险	实际效果
0.1	弱	强 (激进)	较高	★ 最佳
0.5	中	中	中	平衡型
0.9	强	弱 (保守)	低	几乎不学习

注意： β 同时是 implicit reward 的缩放系数 ($r = \beta \cdot \log \text{ratio}$)， β 越大，reward margin 数值本身越大。因此 reward margin 绝对值大并不代表偏好对齐效果更好，需结合实际生成质量综合判断。

5.1 全链路结论

- **Pretrain → SFT**: 模型从"无脑续写器"进化为"指令跟随助手", 学会了对话角色与格式, 但代价是预训练阶段习得的部分知识发生灾难性遗忘;
- **SFT → DPO**: SFT 阶段遗留的"虚拟助手"身份模糊、回答粗糙、维度单一等问题被进一步打磨, 回答更结构化、更符合人类偏好;
- **核心机制验证**: DPO 主要通过**压低 rejected 概率** (而非提升 chosen 概率) 来对齐偏好, 与 KL 约束下的理论预期完全一致;
- **未完全解决**: 两阶段均存在的**复读问题**未被彻底消除, DPO 主要作用在偏好层面, 对生成稳定性属副作用改善而非根本性修复。

六、最大工程阻碍与解决方案

整个炼丹流程并非一帆风顺, 以下是我在"从 0 到 1"过程中遭遇的**最大工程阻碍**以及解决思路。这些问题大多与"算法本身"无关, 却恰恰是工程实践中最耗时间的部分。

阻碍 1: 数据集体量过小

最初我自行从 Hugging Face 上挑选了一份约 400MB 的数据集进行实验, 但与老师确认后发现**数据量远远不足**以训练出有意义的语言模型——以 25M 参数的小模型而言, 数据量至少需要 GB 级别才能让 Loss 真正收敛到合理区间。

解决: 切换为老师推荐的 MiniMind 官方预训练数据集 (127 万条、pretrain_hq.jsonl), 数据量与质量均达标, 后续训练 Loss 曲线表现明显改善。这也让我深刻体会到: **"数据决定上限, 模型决定下限"**在小模型场景下尤其成立。

阻碍 2: 训练环境配置 (最棘手的"非算法"问题)

课程提供的环境配置手册基于**光环云**平台, 而我实际分配到的账号属于 **SCNet 云**平台, 操作流程并不通用, 文档完全失效。

我先尝试在本地 RTX 3090 上启动训练, 但跑了不久整机直接卡死 (显存 + IO 双重压力, 且本地环境的 CUDA / PyTorch 版本与 MiniMind 要求不完全兼容)。

解决: 转向 SCNet 云平台, 自行摸索: ① 创建 Notebook 实例; ② 选择适配 AI 大模型的镜像 (PyTorch + CUDA 全套); ③ 挂载持久化数据盘; ④ 重新配置 conda 环境与依赖。最终在 4090 上稳定跑通整条链路。这一阶段虽然没有写一行模型代码, 却花了将近一天时间——也是整个工程过程中最让人焦虑的部分。

阻碍 3: 训练时长长 + 显存压力 (OOM 隐患)

4090 虽然能跑, 但全流程总耗时不短: Tokenizer 训练 3~5 小时, Pretrain 约 3 小时, SFT + DPO 约 2~3 小时。第一次训练时**反复怀疑自己是否配置错误**、是否陷入死循环, 甚至怀疑是不是数据加载卡住了。

解决: 在训练脚本中加入按步打印 Loss 与 reward margin 的日志 (每 50 步一次),

建立"训练过程可观测性", 确认 Loss 持续下降即可放心等待; 同时通过减小 batch_size (DPO 阶段降至 4) 规避 OOM 风险, 并在保存 checkpoint 时控制频率, 避免磁盘写满。

阻碍 4: DPO 阶段 Loss / Reward Margin 不稳定

DPO 训练初期 reward margin 大幅波动甚至为负, 且 β 参数的选择直接决定模型是否能学到偏好—— β 太大模型几乎不动, β 太小又有遗忘风险。

解决: ① 将学习率从 $1e-5$ 进一步调小至 $1e-6$, 曲线明显平稳; ② 设计 $\beta=0.1 / 0.5 / 0.9$ 三组消融实验, 最终确认 $\beta=0.1$ 在 reward margin 与生成质量改善之间取得最佳平衡。

特别需要指出: $\beta=0.9$ 时 reward margin 数值看起来"最大", 但模型实际几乎没学到东西——这是因为 $r = \beta \cdot \log \text{ratio}$, β 越大 reward 数值天然越大, **reward margin 数值大小不能直接横向比较**, 必须结合实际生成 case 综合判断。这是我在调参中学到的重要经验。

阻碍 5: 复读问题始终未根除

SFT 与 DPO 阶段均观察到模型重复 token 的现象, 尤其在长回答场景下尤为明显。

分析: 这与训练数据长尾分布、模型规模较小 (25.83M)、未引入推理时策略 (如 repetition_penalty / no_repeat_ngram_size) 等因素均有关。

缓解思路: ① 推理时加入 repetition_penalty=1.1~1.3; ② 在更大数据上继续训练; ③ 引入 SimPO / KTO 等改进型偏好对齐算法。受时间所限本次未深入解决, 留作后续优化方向。

阻碍 6: 三方代码同步 (云平台 / GitHub / 本地)

由于 SCNet 云平台、GitHub、本地工作机以及国内网络环境的多重限制, 每次完成实验后都需要进行**三方同步**: 先从云平台下载结果到本地, 再 clone / push 到 GitHub。云平台访问 GitHub 经常超时, 本地推送也因网络抖动经常失败。

解决: ① 在云平台用 scp / Notebook 自带下载功能将结果先拷至本地; ② 本地配置好 SSH key 和加速代理后再统一推送 GitHub; ③ 关键文件 (如 .pth 权重) 通过 .gitignore 排除, 仅同步代码、日志、图表。整个过程虽然繁琐但稳定。

七、总结与反思

本课程的最大收获: 通过这三次环环相扣的作业, 我完整体验了一个语言模型"从无到有"的全过程——从最基础的 token 切分, 到模型从随机权重开始学习语言规律, 再到

通过指令微调与偏好对齐让模型"懂规矩、会聊天"。这种全链路的视角是单看任何一篇论文都无法获得的。

7.1 三次作业的能力跃迁

阶段	核心能力	典型表现	主要问题
Tokenizer	文本 ↔ Token 编解码	压缩率 0.38~0.51	词表偏小，中文压缩率不高
Pretrain	语言建模 / 续写	Loss 7.64 → 2.20	无问答意识，复读严重
SFT	指令跟随	能识别 user/assistant 角色	灾难性遗忘，仍有复读
DPO	偏好对齐	定位清晰、多维论述	复读未根除， β 难调

7.2 个人反思

- 工程问题占了一半时间**：环境配置、网络同步、显存管理、日志监控.....这些"非算法"问题往往比模型本身更耗精力。一个稳定可复现的训练环境，比反复调参更有价值。
- 小模型上的实验同样有意义**：虽然 25.83M 参数远不能与 GPT 系列相比，但 Pretrain → SFT → DPO 的链路逻辑、Loss 收敛规律、对齐机制等核心现象在小模型上已能完整复现，对理解大模型工作机理非常有帮助。
- "看曲线"是一门学问**：DPO 调参中我意识到 reward margin 的绝对值不能直接比较；Pretrain 中 Loss 平台期不代表训练失败；SFT 中 Loss 在 1.95~2.10 间波动是正常现象——这些都需要结合理论才能正确解读。
- 对齐 ≠ 万能**：DPO 改善了回答的结构性与人类偏好性，但并未根治复读、事实性错误等更底层的问题。模型能力的真正提升仍要回到**更大的预训练数据 + 更高质量的 SFT 数据**。

7.3 致谢与代码开源

感谢老师在课程中提供的 MiniMind 框架与高质量数据集，使得"在一台 4090 上完整训练一个语言模型"成为可能。本课程的所有代码与训练日志均已开源至 GitHub：

- 作业一 (Tokenizer)** : https://github.com/qing121201/Tokenizer_Project
- 作业二、三 (Pretrain + SFT + DPO)** : https://github.com/qing121201/MiniMind_Pretrain

—— 报告完 ——

廖佩瑶 · 2022K8009915031 · 2026 年 5 月